

Università degli Studi di Pavia

Bachelor degree in Artificial Intelligence

Planisuss final report - A.Y. 2022/2023

MARCO ZENNARO & FABIO BRUSCHI

1. Planisuss Introduction

The world of Planisuss is an island where the simulation of a small but, at the same time, complex ecosystem take place. Carviz, Erbast and Vegetob are the three entities which populate the world. This project is based on the specifications illustrated in *Planisuss v0.95 - Final exam project* provided by Professor Stefano Ferrari.

1.1. World

The island of planisuss is divided in square units called Cell. Each cell can be either ground or water, ground cells are habitable while water cells are not. Since the world of Planisuss is an island, all the outer cell of the world are water cells. The world is squared and its dimension are calculated in cell units, and the number of cell is provided by NUMCELL; so its size is $\text{NUMCELL} \times \text{NUMCELL}$. Ground cells are “occupied” by vegetob, and can be populated by carviz or erbast. These are the three entities that populate Planisuss, object of discussion of the next section.

1.2. Species

There are a total of three species, two of which are “actively alive”, Carviz and Erbast, while the last one is “passive”, Vegetob.

Vegetob the vegetable specie of Planisuss that grows on ground cells. They are identified by a value between 0 and 100 which describe their density on a specific ground cell.

Carviz and **Erbast** are the animal species of the island. They are characterised by the same attributes, such as energy, lifetime, age, social attitude and differentiate in what they eat: carviz is a carnivore species which eats erbast, while erbast is a herbivore specie and eats vegetob.

Carviz and erbast on the same cell constitute a group, which act, move and think as a single entity. **Prides** are the union of carviz. **Herds** are the union of erbast.

1.3. Simulation

The simulations of Planisuss runs through the temporal unit of **day**. A day on Planisuss consist in a set of actions which simulate the flow of time in a normal ecosystem. The day is further illustrated in section 2.

In order to simulate the world of Planisuss external libraries have been used:

matplotlib: to visualise all the graphs and different screens;

numpy: to arrange and store the world using arrays;

random: to provide randomness and uniqueness in every run of the simulation;

os: to feature a save and load functionality.

1.4. Differences and Motivations

Values of constants, colours, additions and names present in the code could be different from the ones presented in *Planisuss v0.95 - Final exam project*: these are small changes which do not affect the general function of the world of Planisuss. These are choices that we made in order to express our vision of the project and are meant to improve the general experience on Planisuss. Every constant, function, class, choice, is further described in section 4.

2. Phases and algorithms of the simulation

The section is dedicated in explaining how the simulation runs following certain phases and rules. In the following paragraphs all the thought process and the reason related to it will be illustrated. This is an overview on how the code generally works not a detailed review, for code references we suggest looking at section 4. and the code itself, furthermore for a better understanding of cited features we propose reading section 3.

2.1. World creation

A simulation needs a world run in. If no world is loaded in the initial menu screen, then a new world is created. We will explain how the world is generated in a normal way, not considering developer options, since that worlds are pre-generated.

The world object is generated and allocated in the *GlobalVariables* file of the project; this object is a matrix of dimension $\text{NUMCELL} \times \text{NUMCELL}$ which can store an object in each cell. Using a double iteration we assign at each cell of the matrix an instance of our class Cell. So the world is now a matrix that has a Cell object in all of its squares.

While iterating over the matrix we set the boundary cells to be water cells, otherwise the ground cells have assigned a random starting value for vegetob density. Then we proceed with the creation of all the animas in the world. The world matrix is iterated another time, the population of each Cell instance is set to a random chosen animal or group (or none); when a group spawn his density is also chosen randomly between 2 and the maximum spawn allowed by his type (SPAWN_MAX_C, SPAWN_MAX_E). Overall the probabilities of spawn of each entity are the following:

- **No population:** 90 %
- **Carviz:** 3,33 %
- **Erbast:** 3,33 %
- **Pride:** 1,67 %
- **Herd:** 1,67 %

At the spawn of each entity an alphanumeric code, the ID, is assigned to them. This allow for a better understanding of the interaction between entities and for a cleaner output on the console terminal. This code has to be intended as a human tax code, in fact there are similarities. Codes are different for animals and groups. Here an overview of each entity ID:

- **Animals:** *DTX-YN*, where:

D: the day of birth of the animal, an integer number

T: his type or specie, either “C” for carviz and “E” for erbast

X-Y: x and y coordinate of the cell in which the animal is born

N: an alphabetical letter that represent the number of spawn when in a group:

“a” indicate the first animal born in that group and so on; “/” when the animal is not born in a group

Examples:

- 0C32-4\ : is the carviz born in the cell (32, 4) in the day 0 of the simulation, he wasn't in a group at the time he was born
- 12E7-50d : is the erbast born in the cell (7, 50) in the day 12 of the simulation, he was the 4th erbast born in that group that day

- **Groups:** TN , where
 T: the type of the group, either “P” for prides and “H” for herds
 N: the global number in the simulation (not limited to the active ones) of that group spawn, divided for types (there can be two groups number 32 but they must be of different type.
 Examples:
 - P12: is the 12th pride ever formed in the simulation
 - H100 is the 100th herd ever formed in the simulation (do not implies that are active other 99 herd, they might be dead)

Functions and methods related to this section:

- generateWorld
- generateAnimals

2.2. Growing

The vegetob specie is more passive than the other animal species, nonetheless it still modifies over time. Each day of the simulation the first event happen, iterating the world matrix: the vegetob density of each cell increase of a fixed value (GROWTH_RATE). We decided to improve the growing procedure making it more dynamic: whenever an erbast eat a vegetob it will also “remove” part of the roots of the vegetob making the recovery process for the vegetob slower. Numerically this traduces that the vegetob will lose density accordingly to how much the erbast has restores his energy plus a constant factor of 1.33. We then coded the growth of the vegetob to be the next unit when it has been eaten the day before, that is, when the vegetob density is a float number, otherwise to be the fixed value. The minimum of growth increment is 1.

Functions and methods related to this section:

- planisussDay
- increaseVeg
- eatVeg

2.3. Movement

Movement is the second event of a Planisuss day and the first important one. Each animal or group needs to move to survive in this world. Carviz and prides hunt erbast and herd in order to gain energy, while erbast and herd have to run away from nearby carviz or prides and have to find a cell with enough vegetob to eat. Every entity has the same movement possibilities: one cell in every direction, even diagonal, or stay still. However different species chose differently the best cell to move in, calculating different treats and goals. Animals and groups have a memory, which is used to avoid going the previous cell in case no objective is reached. There a deep dive in the thought process of each animal and group:

- **Carviz:** they are scared by prides and large herds (a large herd has 3 or more members) and they hunt for erbast, ideally, but also for small herds. Scanning the surrounding cells, carviz will avoid absolutely the cells that may be reached by his

enemies (note that large herds can kill one carviz, since they outnumber him, but do not hunt carviz, in fact erbast do not gain any energy when killing a carviz); they will search for reachable/safe cells (a cell in a entity movement range that can be safely joined, according to the animal field of view) which contains small herds or erbast hoping they won't move away. If no pray is in range, the carviz will move, if it has energy, in the nearby cell with the most density, knowing that an erbast might land on that cell and fall in the carviz's trap.

- **Erbast:** they are scared by prides and carviz and hunt for the cell with more vegetob density in the area. When an erbast is not chased down by any predator it will move to the cell with the highest density, if he has energy to move, otherwise he will stay in the same cell and eat. If the erbast has a predator in his neighbourhood cells he will run as far as possible from the any, ignoring the starving status and the richest cell; when surrounded by enemies the erbast will try to sneak in carviz's enemy lines, hoping for a swap of cells, in order to stay alive. Furthermore, if the erbast is hunted down and an herd is nearby, the erbast will try to join the herd to be more protected, hoping it doesn't move away.
- **Prides:** they are on top of the food chain and do not fear anyone, except other prides. Prides eat everything they will find but prefer much more herds and then erbast, in fact they will look for these entities in the nearby cells. This "super predator" can allow other carviz to join the group unless its members are very hungry and will kill even their similar to gain energy. If no prey is in the neighbourhood cells prides move to the nearest cell with the most vegetob density (if the average energy is sufficient to move), to set up a trap for incoming herd or erbast.
- **Herds:** herds behave more or less like erbast but, when large enough, they do not fear carviz anymore. They are scared by prides and small herds have also fear of carviz. If prides hunt them down they will run away otherwise, if no enemy is in the area, herds with sufficient average energy can move to the cell with the most vegetob density nearby; if the average energy of the group is low it will stay in the same cell and eat vegetob.

Individual animals choose the next cell by themselves while animals in group can decide to follow the group decision or move away from the group and select a different cell. Each animal in a group compares the group decision based on his necessity and his social attitude, then decide to follow the group or not. Positive group decisions for an individual, for example stay in the same cell and eat when the animal is starting "starving", will increase his social attitude, while discording ones, such as string still for a group when a predator is nearby, will decrease the social attitude of the individual and eventually lead to a different decision by the one.

Functions and methods related to this section:

- planisussDay
- getNeighbours
- nextMove
- chooseBestCell
- eatVeg
- loseEnergy

- `getPopulation`
- `getCoords`

2.4. Struggle and Merging

After animals and group have decided where to move, the next phase is arrange them in their new cells. When more then an entity come to the same cell one between two scenarios happen: the two entity merge into a group or they fight for the position. This behaviour is ruled by the `cellContested` function which actually calls other more specific functions. Here we'll discuss about how the function decide and how the first scenario works; the following paragraph will cover the other scenario.

The function that decide the fate of the struggle phase is the `decideFight` function: it is a simple *if - else* procedural analysis which will return *True* when the two entities will fight and *False* otherwise. Here all the combinations between entities and what they lead to:

- **PRIDE - PRIDE**: this combination will fight only when the difference of average energy between the two groups is greater than a constant, it is very likely to happen
- **PRIDE - CARVIZ**: this combination will fight only when the difference between the current energy of the individual and average energy of the group is greater than a constant, it can rarely happen
- **CARVIZ - CARVIZ**: this combination will fight only when the difference of current energy is greater than a constant, it might happen
- **CARVIZ - ERBAST**: this combination will always fight → return *True*
- **PRIDE - HERD**: this combination will always fight → return *True*
- **CARVIZ - HERD**: this combination will always fight → return *True*
- **PRIDE - ERBAST**: this combination will always fight → return *True*
- **ERBAST - ERBAST**: this combination will never fight → return *False*
- **ERBAST - HERD**: this combination will never fight → return *False*
- **HERD - HERD**: this combination will never fight → return *False*

If the two entities won't fight they will eventually merge into a single group with the following rules depending on the two entities:

- Two groups: a new group is created and the members of the two groups are joined, disregarding the maximum capacity of a group
- One group and an animal: the animal will join the group if there is enough space, otherwise he will have to move in another cell
- Two animals: a new group is founded with members the two animals

Functions and methods related to this section:

- `planisussDay`
- `cellContested`
- `decideFight`
- `mergeEntities`
- `addAnimal`

2.5. Fight system

When animals decide to fight for the same cell, whether they do not find an agreement in sharing the same cell or they are simply hungry, the fight function rule this phase. Every fight is unique and different in all of his aspects from any other battle; this result is achieved creating a complex combat system that do rely on many independent factors, advantages and power relations, but also rely on probability. This choice is clearly made to give to every animal the opportunity to win a fight if in the right condition and with a bit of luck. Only few fight can be decided a priori with this system. Here are the rules for the fights:

- Individual fights and groups fight are taken in **duel(s)**, that can be either Carviz vs Carviz or Carviz vs Erbast.
- Group fight are so structured: a member of a group fight in a duel a member of the other group the winner come back in his group; the procedure is repeated until no members of a group are left.
- Duels are based on probability and each duel is won with a best of 3 (Bo3) fight with the opponent: each probability is chosen three times and one point is assigned in every roll; the animal with at least two wins in the fight wins the duel.
- The Bo3 system has been chosen to avoid luck situations, you have to win two times by luck to win the duel (and is much harder)
- Odds of victory aren't equal between the two species and depends by the energy of the two animals.
- Carviz are much stronger than erbast and will most likely win the duel, however erbast can win the duel when the difference in energy of the two animal is very big.
- The only scripted battles are the following and have the indicated result:
 - ✦ *Pride against carviz*: the carviz will always die and the members of the group gain energy
 - ✦ *Carviz agains large pride*: the carviz will always die and no erbast of the herd will gain energy more over they will lose a bit of energy for the "fight"
 - ✦ *Pride against erbast*: the erbast will always die and the carviz member of the pride will gain a little amount of energy
 - ✦ *Pride against one herd with no more than the 130% the population of the pride*: the herd is always killed entirely with no fight and each carviz of the group will gain energy

Here is explained the probability distribution for the duels:

- **Carviz vs Carviz** (they fight on the same level):
 - The energy difference between the two carviz is calculated
 - The probability of the first carviz to win a match of the duel is: $50 + (\text{energy difference} / 2 \text{ and rounded to the unit}) \%$
 - The probability of the other carviz to win a match of the duel is: $100\% \text{ minus the probability of winning of the first carviz}$
 - Then the Bo3 duel take place
 - *Examples*:
 - ▶ c1 has an energy value of 60, while c2 of 45. The energy difference is $60 - 45 = 15$, then the probability of winning for c1 is $50 + 15/2 = 50 + 7.5 = 50 + 8 = 58\%$, while c2 has a probability of 42%

- ▶ c1 has an energy value of 20, while c2 of 72. The energy difference is $20 - 72 = -52$, then the probability of winning for c1 is $50 + (-52)/2 = 50 - 26 = 24\%$, while c2 has a probability of 76%
 - Ultimately this means that an advantage in energy transpose to higher probabilities of victory but no result can be certainly determined
- **Carviz vs Erbast** (do not fight with the same odds, since carviz are stronger):
 - The energy of the erbast is initially divided by 2 (and rounded to the unit)
 - If the energy of the carviz is still less then the energy of the erbast, the carviz increase its energy by 1.5 times
 - The energy difference is computed. If the energy difference is a negative number, it is multiplied by a factor of 0.8, otherwise is multiplied by a factor of 1.2
 - The probability of the carviz of winning a match of the duel is: $50 + (\text{energy difference divided by 2 and rounded to the unit}) \%$
 - The probability of the erbast to win a match of the duel is: 100% minus the probability of winning of the carviz
 - *Examples:*
 - ▶ C has an energy value of 60, while E of 45. The energy of E is firstly halved, so it becomes $45/2 = 22.5 = 23$. The energy difference is $60 - 23 = 37$, since $37 > 0$ then the energy difference is set to be $37 * 1.2 = 44$. Ultimately the probability of winning for C is $50 + (44/2) = 72\%$ while E has a probability of winning of 28%
 - ▶ C has an energy value of 40, while E of 70. The energy of E is firstly halved, so it becomes $70/2 = 35$. The energy difference is $40 - 35 = 5$, since $5 > 0$ then the energy difference is set to be $5 * 1.2 = 6$. Ultimately the probability of winning for C is $50 + (6/2) = 53\%$ while E has a probability of winning of 47%
 - ▶ C has an energy value of 20, while E of 80. The energy of E is firstly halved, so it becomes $80/2 = 40$. Since $40 > 20$, then the energy of C becomes $20 * 1.5 = 30$. The energy difference is now $30 - 40 = -10$, since $-10 < 0$ then the energy difference is set to be $-10 * 0.8 = -6$. Ultimately the probability of winning for C is $50 + (-6/2) = 46\%$ while E has a probability of winning of 54%
 - This distribution allow for a dominance of the carviz over the erbast, since are stronger creatures, but when a large difference of energy is faced, the probabilities almost levels up.

Functions and methods related to this section:

- planisussDay
- fight
- averageEnergy
- kill
- gainEnergy
- loseEnergy

2.6. Food chain and Energy management analysis

The Planisuss world is can be really complex, such as in the fight system, but watched from the distance it is really simple and has a very few main rules:

- Vegetob can only be eaten by erbast;
- Generally carviz eat erbast, however very strong erbast might wins against very weak carviz by survival instinct;
- Carviz can be eaten only by other carviz;
- Prides can be beaten only by other prides;
- In very remote and unlikely cases there is the opportunity that a small pride with very weak members might be killed by a very large herd with very strong members;
- Herds can kill isolated carviz if they outnumber him, but only by survival instinct.

With this type of food chain the energy gain and lose system is so structured:

- **Erbast:**
 - Gain energy only by eating vegetob
 - Lose energy fighting and survive against carviz and move to other cells
- **Carviz:**
 - Gain energy by eating both carviz and erbast, however they gain more energy eating erbast because they have to fight less
 - Lose energy by moving
 - Do not lose nor gain energy if they stay still

The management of the energy happens both in the fight function and in the nextMove method depending on the action chosen or the outcome of duels. Generally we think that this system allows for a fair interaction between the different species in the world, for example a weak carviz might prefer staying still and waiting for food rather than move and lose energy; this is better for a carviz because it does not lose energy doing nothing, that is, for us, like having a timed bomb on top of the carviz head. The values of energy gained and lost by each cell are ruled by constants in the settings file, allowing for an easy access and modification. In this page is also present a constant that is the percentage at which different animals and groups absolutely need food.

When the energy of an animal reaches 0 the animal dies and won't come alive.

The energy attributes of an animal are 2, set randomly at his birth: the maximum energy possible and the current energy. While the current energy can be increased or decreased gaining and losing energy, the maximum energy is fixed and modifies over time according to the age of the animal: old animals have a lower maximum energy and therefore will start starving sooner than a just born animal. The maximum energy value is updated in the increaseDayLived method.

Functions and methods related to this section:

- planisussDay
- eatVeg
- gainEnergy
- loseEnergy
- increaseDayLived
- birth
- isDead

2.7. New generations

At the end of the day all the entities survived sees their age increased by one. When an animal has reached an age equal to his pre-determined lifetime, the death will naturally come, the fate will take away the animal from the world but he has one more thing to do: generate children. This is the final objective of a Planisuss' animal. The offspring are so generated:

- The number of children is randomly determined between a maximum and a minimum fixed constant; one animal will at least generate one child at the end of his life but the maximum number depends on the specie. We decided to take this approach instead a fixed number of children to avoid creating an overpopulation and differentiate even more erbast and carviz (the first specie have a maximum number of children grater than the second specie)
- The starting energy, the maximum energy and the lifetime of the offspring is randomly assigned at the birth. Since we chose to fix a maximum amount of energy for each individual, aiming for a more balanced ecosystem (with this approach there's not the possibility of creating the "super carviz" with almost infinite energy), we could not divide the energy of the dying carviz between the children because future offspring's would have been too weak for survive more than a butterfly
- All the other properties are set to 0

Animals who born from a member of a group might spawn in a different nearby safe cell if the group is too crowded and has reached the maximum density, otherwise, animals generally spawn in the same cell of the father. When an alone animal die it will create a new group if he generates more then one child.

Functions and methods related to this section:

- planisussDay
- birth
- moveAnimals
- addAnimal

2.8. Other details, longevity of the simulation, future updates

All the phases of the day are ruled by the planisussDay function, which use an auxiliary matrix called world_help to store momentarily the moving animals, then let them fight merge, die and reproduce and ultimately, with a last iteration, the animals are brought back in the world main matrix.

This simulation can runs for many days, however changing, adjusting and experimenting with different values in the settings file might lead to better and longer simulations. We decided to focus more on the UI/UX part of this project spending more time on studying, implementing and presenting different graphs, features and gimmicks of the matplotlib library. We decided to limit the number of world that the simulation can run and slowing it down to really appreciate all of its features but also this settings can be changed.

Futures updates may be released and can include:

- Develop a better selection process of the next cell for every animal and group, maybe implementing an actual artificial intelligence rather than a simple (but still

effective) if - else selection. (This result might be achieved at the end of the bachelor degree, but is the most relevant one)

- A new memory system (linked to the first point)
- Further graphs that can displays even more informations
- The ability of changing the simulation with the same UI and controls, giving the opportunity to load a compatible code
- Adding a better menu to changes the constant of the simulation when it is running
- Adding species or natural events (but would it still be planisuss?)

Functions and methods related to this section:

- `planisussDay`

3. Data visualisation, additional features and controls

All the windows and graphs are created using the matplotlib library. Many and different functions, classes and instances of this complex library have been used to show its utility in various scenarios.

3.1. World visualisation choices

The world is updated and displayed at each day in a window. Each cell is actually represented by a colored square in a graph without axes. The color of each square is related to the content of each cell which correspond to a specific RGB value. The color value of each cell is stored inside the *color* attribute of the instance of the class *Cell* present in that cell; then it is transposed to a matrix of RGB values using a function and, eventually, the matrix is plotted using the *imshow* method. Colours are so assigned:

Color	RGB value	Entity represented
Yellow	[1, 0.85, 0]	Water
Olive green	[0.33, 0.42, 0.18]	Vegetob at the max density
Green (different hue)	[0, *, 0]	Vegetob, * depends on the density
Red	[1, 0, 0]	Carviz
Pink	[1, 0.37, 0.6]	Pride
Blue	[0, 0, 1]	Erbast
Light blue	[0.20, 0.48, 0.98]	Herd
Brown	[0.4, 0.24, 0.08]	“Ground”, when vegetob density is too little

The use of different colours for different entities depends both on visual and functional choices: a colourful world appear more alive, also, for the animals, it was not possible to use the intensity of the color of the cell as an indicator of the density of its population: cells with only one individual would have been too dark and mix together, so we opted for a fixed color for single individual (but and red) and group (pink and light

blue). Furthermore, we chose yellow as the color of the water on Planisuss because this world has a different water chemistry and we thought it would be more intuitive and visually distinctive to chose blue, alongside with red and green, as color for the species. Olive green and brown are used to separate the vegetob density and let the ground cells appear less flat. Finally we chose red for the carnivore species, because it recall the color of blood, blue for the herbivore species, because it recalls the calm of the ocean and so the non hostility, and green for the vegetob, because it recalls the plants of our world. Consequently a shade of red (pink) and blue (light blue) is used to distinct between prides and herds. This color choice is consistent throughout all the graphs.

3.2. Starting Menu

Before the simulation starts in the main screen, another window will appear: this is the initial menu. Here a welcome text is shown, below it the controls are listed. At the bottom a dynamic text will show whether a save file have been found in the directory of the program and eventually give the option to load it. From this menu is also possible to access developer options. The user can start a loaded world or a new simulation simply closing this window, immediately the animation will start.

3.3. Simulation window

The simulation runs through an instance of the class FuncAnimation in the main program. In this window is possible to view state of the simulation, *paused* or *running*, and also the current day of the simulation. It is possible to stop and resume the animation by pressing [SPACE]. When the simulation is paused is possible to jump between other screens, that show different plots of various information of the world using keyboard commands. Using [<] (left arrow) and [>] (right arrow) is possible to move between the saved days and see the precise value of a specific plot in that day.

3.4. Vegetob graph

Can be accessed using the [V] key when the simulation is paused.

It shows the global average density of vegetob present in the world during days. Simple but effective.

3.5. Population graph

Can be accessed using the [P] key when the simulation is paused.

In this screen there are four plots that shows different informations, *over days*, about the two animal species of Planisuss. Here what they illustrate and how to interpret them.

- **Upper-left:** it shows the total number of carviz (represented by square mark) and the number of carviz that do not belongs to any pride, that is, alone (represented by a dot mark). Here the user can watch, fixed the number of total carviz, if many of them prefer live or survive better alone than in a group (when the graph has high values) or the other way around.
- **Upper-right:** it shows the total number of erbast (represented by square mark) and the number of erbast that do not belongs to any herd, that is, alone (represented by a dot mark). It has the same reading of the previous graph.

- **Bottom-left:** it compares the number of total carviz that belongs to prides (red) and to the number of prides (pink). Here the user can monitor the average density of prides; it increase if the two bars are widely separated and vice versa.
- **Bottom-right:** it compares the number of total erbast that belongs to herd (blue) and to the number of herd (light blue). The same trend of the previous graph can be observed here.

Its worth mentioning that the first two and last two graph shares the same y-axis, this allows for a quick cross-analysis between informations of different species.

3.6. General graph

Can be accessed using the [G] key when the simulation is paused.

This screen can be divided in two sections:

- **Left:** one graph that shows the total number of carviz, erbast, pride, herd and the global vegetob density all together over the saved days. It is useful to rapidly keep track of the main interactions of the world.
- **Right:** three bars, the first two divided for each animal species, which visually express the importance of the respective values compared to the others in a specific day. This helps to compare immediately all the value of a single day

This screen offers many informations condensed in the same place, it is a perfect page to have an analysis at a glance of the simulation. To see a lot of datas other provided screens might be more useful.

3.7. Pause, Rewind and forward

To pause the simulation press the [SPACE] key.

When the simulation is paused, it is possible to use the [<] (left arrow) and [>] (right arrow) keys to navigate between the saved days. This change the day selected to show the world (when in the world screen) but it also reflects in all of the various graph and save screen of the program. It is important to remember that is possible to rewind the world until one of this three boundaries are reached: day 0, limit of number of saved days, starting day of a loaded world. To make an example, if the user stop a brand new simulation on day 20 and the limit of storage is 15 days, day 5 will be the last day that is possible to rewind. Also, the day in which the user stop the simulation is the maximum day to which the user can move forward.

During a paused status, pressing the [SPACE] key from any screen will automatically resume the simulation from the day it was previously stopped.

3.8. Save and load

Load of previously saved world is accessible in the starting menu.

The save screen can be accessed using the [x] key when the simulation is paused.

The option of loading a previously saved world is prompted in the starting menu when a save is actually found. The user can press the [L] key to load the world, its respective day and other informations, then close the window to “resume“ the simulation.

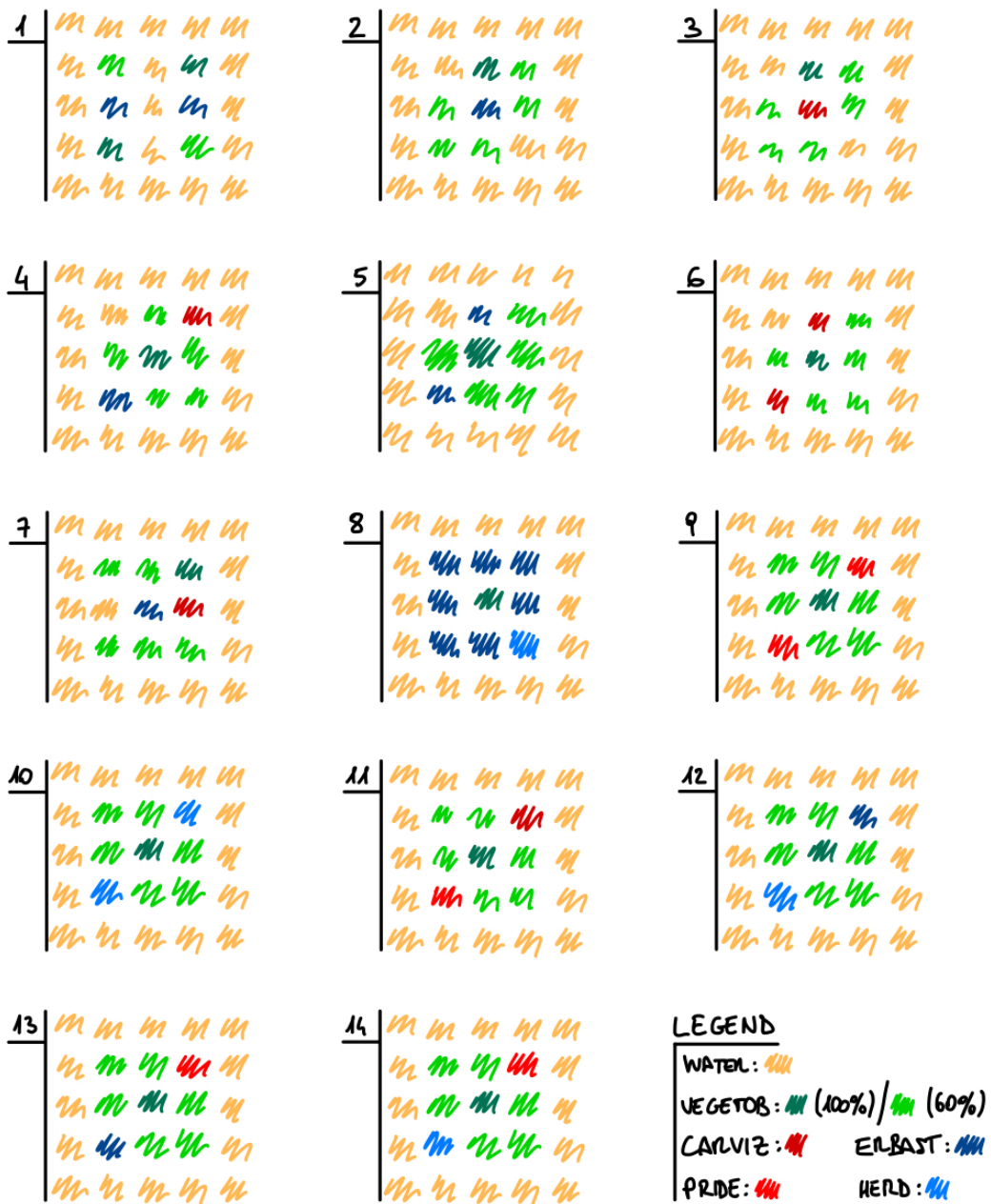
Saving a world status in a specific day can be accessed in the homonym menu in the main window of the simulation. Here the user can chose which of the day simulated he/she wants to save, then confirm the choice. Two files are created (or overwritten, if

already present) in the directory of the project: one .npy that store the world and the other .txt that save day and data informations about the world.

Is also possible to save a .png file of whatever screen using a built-in function, pressing the [S] key.

3.9. Console outputs and developer options

The developer options consist in a different way of generating the world, used by us to test various features. It allows to pick a pre-created world among 14 by type the corresponding number in the console. Disclaimer: this is a fancy feature and requires that NUMCELL = 5 and the frame value in the FuncAnimation class is 2 or 3 in order to be useful. Here a scheme of the developers worlds, from 1 to 14:



All of the useful information about classes are printable: their `__str__` method have been overwrite and is possible to see them properly in the console. The related classes

are: Cell, Animal, Group, DoublyLinkedList. In the code there are comments of the kind “ # print(*data*) ”, it is possible to remove the “ # ” and visualise the output of the data.

3.10. Commands

Here the list of commands:

Key	Command
[SPACE]	Pause and resume the animation
[V]	Show/hide the vegetob graph (at simulation paused)
[P]	Show/hide the populations graphs (at simulation paused)
[G]	Show/hide some general graphs (at simulation paused)
[W]	Show the world (at simulation paused)
[X]	Show the save screen (at simulation paused)
[<] / [>]	Move back and forward between the saved days (at simulation paused)
[L]	Load a previous save of the world (starting screen)
[D]	Activate developer options (starting screen)
(close)	Start simulation (starting screen)
[S] (built-in)	Save the current output image into a .png image

4. Code details and further explanations

In the following paragraphs there are all the details regarding each file, class, function and constant of the program.

4.1. Main

This file include the main function.

The main function initially calls the startingMenu function. Depending on the values of variables in the *GlobalVariables* file, the main function can either generate a new plain world, generate a developer world or generate no new world (this because an old world have been loaded). Then it create and show the main windows in which the animation runs.

4.2. Classes

Here a list of the classes that runs the world, with details about attributes and methods.

Cell

Description	
The class Cell has one instance in each cell of the $\text{NUMCELL} \times \text{NUMCELL}$ matrix of the world of Planisuss. It is responsible for the organization of color, vegetob, animals and group, coordinate system and water.	
Attributes	Description
__Xcoordinate	Is the x coordinate of the cell in which the instance is located
__Ycoordinate	Is the y coordinate of the cell in which the instance is located
__vegetob_density	Float value that represent the density of the vegetob species in that cell
isWater	Bool value that determine if the cell is a water cell
population	An instance of another class, can be carviz, erbast, pride or herd
__color	List containing the three RGB values of the cell
Methods	Description
__init__	Set the coordinates of the cell and give a random value to the vegetob density attribute
getCoords	Return the x and y coordinate of the cell in a tuple (x, y)
setWater	Set the current cell as a water cell
increaseVeg	Increase the density of vegetob in the cell
decreaseVeg	Decrease the density of vegetob in the cell
defineColor	Compute the color of the cell based on the type of cell and its population
setColor	Set a specific color for the cell
getColor	Return the color array of the cell
setPopulation	Set a given population in the cell
getPopulation	Return the current population of the cell
setVegetobDensity	Set a given value as __vegetob_density
getVegetobDensity	Return the value of __vegetob_density
getNeighbours	Return the surrounding cells and itself
__str__	Allows for a cleaner output of the informations about the cell

Animal

Description	
The animal class has attributes and methods that are common on both carviz and erbast. Specific attributes and methods are in the respective subclasses	
Attributes	<i>Description</i>
id	Unique ID of the animal
energy	Integer value for the current energy of the animal
max_energy	Integer value for the max energy that the animal can have
lifetime	Integer value that set the maximum lifespan of the animal
age	Integer value that express how many days the animal has lived
inAGroup	Bool value that specify whether the animal in a group
soacial_attitude	Value between 0 and 10 that indicate how much the animal likes the group is in. Value 0 indicate he is not in a group
__memory	List where the previous cell positions are saved
specie	String that identify the specie of the animal
Methods	<i>Description</i>
__init__	Set the ID of the animal
increaseDayLived	Increase the age of the animal and reduce his max energy
isDead	Return a bool value that represent if the animal has 0 energy, that is, he is dead
memorySize	Return the integer value of the length of the memory
getMemory	Return the entire memory list of the animal
getLastMemory	Return the last item in the memory
addMemory	Add a cell in the last position of the memory and, if the memory is too long, it remove the first element
kill	Set the energy value of the animal to 0
checkDeath	Return None if the animal is dead or the animal if he is alive
__str__	Allows for a cleaner output of the informations about the animal

Carviz (subclass of Animal)

Description	
The Carviz class is a subclass of Animal. It adds some specific methods, different from the one in the Erbast class	
Attributes	<i>Description</i>
^	
Methods	<i>Description</i>
__init__	Set the ID and specify the “carviz” value of the attribute specie
loseEnergy	Remove a given integer value from his energy
gainEnergy	Add a given integer value to his energy
nextMove	Return a list containing a tuple which contains himself and the coordinates of the next cell the carviz will go
chooseBestCell	It calculates and return the best cell to occupy from the given ones (a neighbourhood of the current cell)
birth	Give birth to a new offspring of carviz when the age of the carviz has reached his pre-determined lifespan

Erbast (subclass of Animal)

Description	
The Erbast class is a subclass of Animal. It adds some specific methods, different from the one in the Carviz class	
Attributes	<i>Description</i>
^	
Methods	<i>Description</i>
__init__	Set the ID and specify the “erbast” value of the attribute specie
loseEnergy	Remove a given integer value from his energy
gainEnergy	Add a given integer value to his energy
eatVeg	It calls the gainEnergy method and decrease the vegetob in the cell
nextMove	Return a list containing a tuple which contains himself and the coordinates of the next cell the erbast will go
chooseBestCell	It calculates and return the best cell to occupy from the given ones (a neighbourhood of the current cell)
birth	Give birth to a new offspring of erbast when the age of the erbast has reached his pre-determined lifespan

Group

Description	
The Group class moderate the life of multiple animals as one. Decision taken by the group are weighted on the characteristics of its members	
Attributes	Description
id	The unique ID of the group
population	List containing all of the animals instances member of the group
__memory	List containing the previous visited cells
type	String value that determine the type of group, either pride or herd
Methods	Description
__init__	Set the ID of the group and its population. It also change the value of inAGroup attribute to True and social_attitude attribute to 3 of each member of the group
populationSize	Return the integer value of the length of the population list
averageEnergy	Compute and return the value of the average energy in the group
addAnimal	Add a given animal object to the list of members of the group
removeAnimal	Remove a given animal from the member list
increaseDayLived	Calls the homonyms method for each animal member
memorySize	Return the integer value of the length of the memory list
getMemory	Return the memory list of the group
getLastMemory	Return the last cell object in the memory list
addMemory	Add a given cell to the memory list
checkDeath	Check if the members of the group are still alive. In the end it returns either None, if nobody is alive; one animal object, if he is the last survivor; itself, if the group is still intact
birth	Calls the birth method for each members and eventually return the offspring that cannot be added to the group
__str__	Allows for a cleaner output of the informations about the animal

Pride (subclass of Group)

Description	
The Pride class is a subclass of Group. It adds some specific methods, different from the one in the Herd class	
Attributes	<i>Description</i>
^	
Methods	<i>Description</i>
nextMove	Return a list containing tuple of itself and the coordinate it will go and, for each member of the group, another tuple which contains the animal himself and the coordinates of the next cell he wants go when he wants to leave the group
chooseBestCell	It calculates and return the best cell to occupy from the given ones (a neighbourhood of the current cell)

Herd (subclass of Group)

Description	
The Herd class is a subclass of Group. It adds some specific methods, different from the one in the Pride class	
Attributes	<i>Description</i>
^	
Methods	<i>Description</i>
nextMove	Return a list containing tuple of itself and the coordinate it will go and, for each member of the group, another tuple which contains the animal himself and the coordinates of the next cell he wants go when he wants to leave the group
chooseBestCell	It calculates and return the best cell to occupy from the given ones (a neighbourhood of the current cell)

4.3. Functions

Here a list of all the program's functions with their description and specifics (note that the "calls to other functions/methods" exclude calls at matplotlib, numpy, random and os libraries, it only include the ones that we write brand new.

planisussDay

Description: It is the main function that rule the day on Planisuss. Initially a new empty supporting matrix of the same sizes of the world is created. Secondly, the vegetob grow and all the animal species in the world move to a new cell in in the assisting matrix; if more animals insist on the same cell they may merge or fight. Then, some controls are made, such as if an animal is dead during a fight, if the group is too crowded and a new member can't enter and so it has to change cell. At the end all of the survivors have their age increased and, eventually, if they reach the end of their lifespan, might generate a new offspring. Ultimately all the animals ready for the next day are brought back onto the main world matrix.

Arguments:

None

Return values:

None

Calls to other functions/methods:

- increaseVeg
(method)
- moveAnimals (function)
- checkDeath (method)
- increaseDayLived (method)
- birth (method)
- setPopulation (method)

moveAnimals

Description: This function is responsible to put each animal and group in the the correct cell of the assisting matrix. It calls the method responsible to calculate, for both animals and groups, the next move, which returns the coordinates of the next cell. If a cell has reached the maximum amount of entities, so some animal is not able to join the group, another call to this function is made, including the parameter exclude_cell, in order to avoid that animals do not go in the "full cell".

Arguments:

- cell: the current cell of the world
- entity: the animal or group that will chose the next cell
- world_help: the assisting matrix where put the animals/groups
- exclude_cell (default=None): cell to avoid for an animal or a group

Return values:

None

Calls to other functions/methods:

- nextMove
(method)
- cellContested (method)
- moveAnimals (function)

cellContested

Description: This function is created in order to give a better readability of the code. It simply call the decideFight function and, based on the returned value, will call the fight or the mergeEntities function and return the its values.

Arguments:

- entity1: the first animal or group
- entity2: the second animal or group

Return values:

- Tuple of two objects (either one animal/group and None or two animal/group)

Calls to other functions/methods:

- decideFight (function)
- fight (function)
- mergeEntities (function)

mergeEntities

Description: It merges the two entities provided: if there is a group and an animal it adds the animal to the group, if there are two animals it creates a new group and if there are two groups it merges the two groups.

Arguments:

- entity1: the first animal or group
- entity2: the second animal or group

Return values:

- Tuple of two objects (the final group after merging, either None or the extra entity that couldn't have been added to the group because full)

Calls to other functions/methods:

- addAnimal (method)

fight

Description: This function makes the two given entities passed and return the winning one, also computing the energy gained-lost procedure. The fight system has already been covered in section 2.

Arguments:

- entity1: the first animal or group
- entity2: the second animal or group

Returning values:

- One object (the winning animal or group)

Calls to other functions/methods:

- kill (method)
- gainEnergy (method)
- loseEnergy (method)
- isDead (method)
- addAnimal (method)
- populationSize (method)

decideFight

Description: This function is responsible for deciding whether the two provided entities have to fight for a contested cell. Given two entities it will return either true or false based on some parameters. The fighting system has already been covered in section 2.

Arguments:

- entity1: the first animal or group
- entity2: the second animal or group

Returning values:

- Bool (entities will fight if true and will merge together if false)

Calls to other functions/methods:

- averageEnergy (method)
- populationSize (method)

generateWorld

Description: This function is responsible of the creation of the world, including the surrounding water and the initial spawn of the animals and vegetob. It also set the global_day parameter to zero when starting a new world. In each cell of the GlobalVariables.world matrix it put an instance of the class Cell, responsible of storing informations about: whether that cell is a ground or water cell, the population of that cell, the vegetob density, the color of the cell. Then a call of the generateAnimal function is made.

Arguments:

None

Return values:

None

Calls to other functions/methods:

- generateAnimal (function)

generateAnimal

Description: This function makes animals and groups of animal with different density randomly spawn across the world.

Arguments:

None

Return values:

None

Calls to other functions/methods:

- setPopulation (function)

worldConverter

Description: this function converts the world into a matrix of RGB-values that can be plotted in a graph. Additionally, during the process of color extrapolation for each cell, it also save some informations about the species. Keys to access the data_save stored informations:

- "carviz_tot"

- "erbast_tot"
- "carviz_indiv"
- "erbast_indiv"
- "carviz_group"
- "erbast_group"
- "pride_tot"
- "herd_tot"
- "global_vegetob_densisty"

Arguments:

None

Return values:

- One object (the $\text{NUMCELL} \times \text{NUMCELL} \times 3$ matrix of RGB-values)

Calls to other functions/methods:

- getColor (method)
- getVegetobDensity (method)
- populationSize (method)
- addDayData (method)

simulation

Description: this is the function that is passed into the FuncAnimation class. Here the planisussDay function is called at each iteration of the animation, along side the increment of the global_day of the simulation. It is also responsible of plotting the converted RGB world onto a graph.

Arguments:

- i: the iteration variable used by FuncAnimation to run the animation

Return values:

None

Calls to other functions/methods:

- planisussDay (function)
- worldConverter (function)

updateScreen

Description: this function is responsible for updating the output image of the main window when the simulation is stopped. It can plot: the world and the rewinded worlds, the general graphs, the save screen, the vegetob graph, the population graphs. At each call the function start by clearing the screen and ultimately draw the output.

Arguments:

None

Return values:

None

Calls to other functions/methods:

- getData (method)
- getFirst (method)
- getNext (method)

- `getDay` (method)

navigateAnimation

Description: it allows to read the input of the keyboard when we are in the main window. According to the key pressed it call the `updateScreen` function to eventually change the screen. The supported inputs are listed in the command paragraph of section 3.

Arguments:

- `event`: the input of the keyboard

Return values:

None

Calls to other functions/methods:

- `getLast` (method)
- `updateScreen` (function)
- `getData` (method)
- `size` (method)
- `getPrev` (method)
- `getNext` (method)

startingMenu

Description: this function is in charge of the administration of the initial menu, before the simulation starts. It search for saved data and show a welcome screen.

Arguments:

- `event`: the input of the keyboard

Return values:

None

Calls to other functions/methods:

None

loadMenu

Description: this function allow to refresh the initial screen when determined key are pressed. It show the status of the load or the developer options (both if selected).

Arguments:

None

Return values:

None

Calls to other functions/methods:

None

worldToString

Description: Responsible of printing in clean way informations about each cell of the world each day.

Arguments:

None

Return values:

None

Calls to other functions/methods:

None

generateWorldDEV

Description: Differs from the standard function in calling the generateAnimalDEV rather than the normal one.

Arguments:

None

Return values:

None

Calls to other functions/methods:

- generateAnimalDEV (function)

generateAnimalDEV

Description: Differs from the standard function in letting the user chose one of 14 pre-created worlds, based on NUMCELL = 5.

Arguments:

None

Return values:

None

Calls to other functions/methods:

- setPopulation (function)
- becomeWater (function)

4.4. GlobalVariables

A table of the “global” variables that runs the world (in the next page).

Variables	<i>Type</i>	<i>Description</i>
DEV	Bool	True if the developer option are activated
world_loaded	Bool	True if a world has been loaded using the starting menu
world	Object	The main world where the simulation is held and the processing done
group_count_p	Integer	Keep track of the number of global prides spawned during all the days of the simulation
group_count_h	Integer	Keep track of the number of global herds spawned during all the days of the simulation
current_day	Integer	Is the day in which the simulation has stopped or have been rewinded/forwarded
global_day	Integer	Is the ultimate day in which the simulation has run, the day from which the simulation will be resumed
data_save	Object	It is a doubly linked list that save all of the global informations about species throughout the days
image_save	Object	It is a doubly linked list that save the image of the world throughout the days
world_save	Object	It is a doubly linked list that save the status of the world throughout the days
world_running	Bool	True when the simulation is running False the simulation is paused
anim	Object	Variables that store the instance of FuncAnimation that allows to run an animation
current_data	Bool	The data stored at the current_day
current_image	Bool	The image of the world stored at the current_day
current_world	Bool	The world stored at the current_day
save_found	Bool	True if a previous save have been found in the directory of the project
world_screen	Bool	True if the window output displays the world screen
vegetob_screen	Bool	True if the window output displays the beget graph screen
population_screen	Bool	True if the window output displays the population graphs screen
graph_screen	Bool	True if the window output displays the general graphs screen
save_screen	Bool	True if the window output displays the world screen

4.5. Doubly Linked List

We took the code studied during the second module of the computer programming course and modify it to get a doubly linked list data structure, which allow us to store and navigate quickly between data and days.

Class DayData

Description	
The DayDate class is the smaller unit that allows the doubly linked list to work. This base element is slightly modified than the standard "Node" in order to give us more flexibility when storing data. In fact it allows for two entries: one for the data itself and another one for the corresponding day	
Attributes	Description
next	The next data saved
prev	The previous data saved
day	The day of the saved data
data	The data stored
Methods	Description
__init__	Set the day and the data to store
setData	Store a given data
getData	Return the stored data
setDay	Set a given day
getDay	Return the stored day
setNext	Set the given object as the next node of the doubly linked list
getNext	Return the next object
setPrev	Set the given object as the previous node of the doubly linked list
getPrev	Return the previous object
__str__	Allows for a cleaner output of the day and informations stored

Class DoublyLinkedList

Description	
The DoublyLinkedList class allow to create a linked list data structure to store useful data that can be later processed and shown on screen in graphs. Note that not every method of a standard doubly linked list is implemented but only the most useful for our project	
Attributes	Description
head	The first element of the list
tail	The last element of the list
Methods	Description
__init__	Initialise head and tail to None
isEmpty	Return a bool value that represent whether the list is empty or not
addDayData	Allows to store a given day and given data informations in a new DayData object at the end of the list
addDataInDay	Allow to substitute the previously stored data with new given ones in a specific day provided
size	Return the integer number of the
searchDay	Search a given day and return it if it has been found, alternatively it returns None
getFirst	Return the first object DayData in the list, that is, the head
getLast	Return the last object DayData in the list, that is, the tail
removeHead	Remove the current head of the list and set the second element, if present, as the head
remove	Remove the object corresponding at the given day (if found)
__str__	Allows for a cleaner output of all the informations stored in the list

4.6. Settings

A table of the global constant of the project (in the next pages)

Constants	<i>Description</i>
NUMCELL	Dimension of a side of the square world of Planisuss
COLO_WATER	RGB color of water
COLOR_MAX_VEGETOB	RGB color of vegetob when they have reached the maximum density in the cell
COLOR_CARVIZ	RGB color of a carviz
COLOR_PRIDE	RGB color of a erbast
COLOR_ERBAST	RGB color of a pride
COLOR_HERD	RGB color of a herd
COLOR_EARTH	RGB color of the cell when the vegetob density id too little
PROB_WATER_SPAWN	Constant that express maximum divisor of the probability in which a single water cell can spawn (the real probability is 1/PROB_WATER_SPAWN)
MAX_DAY_STORAGE	The maximum numbers of days that can be stored in every memory of every variable/instance
MIN_ENERGY	The minimum energy that an animal can spawn with
MAX_ENERGY	The maximum energy that an animal can spawn with
MIN_AGE	The minimum lifespan that an animal can spawn with
MAX_AGE	The maximum lifespan that an animal can spawn with
MAX_SOCIALNESS	The social value for an animal
MAX_GROUP_SIZE	The maximum size groups can be
MOV_ENERGY_LOSS_C	The quantity of energy loss during the movement to another cell for a carviz
CAR_EAT_ENERGY_GAIN	The quantity of energy gained by a carviz when eating another carviz
ERB_EAT_ENERGY_GAIN	The quantity of energy gained by a carviz when eating an erbast
FIGHT_ENERGY_LOSS_C	The quantity of energy loss by a carviz when fighting another carviz
SPAWN_MAX_C	The maximum number of carviz that can spawn in a Pride
PERC_STARVING_C	The percentage of the self energy at which a carviz start “starving” and needs absolutely food

Constants	<i>Description</i>
PERC_STARVING_P	The percentage of the average energy between the members of the group at which a Pride start “starving” and search absolutely for food
MAX_BIRTH_C	The maximum number of offspring that a carviz can generate at the end of his life
MOV_ENERGY_LOSS_E	The quantity of energy loss during the movement to another cell for an erbast
VEG_EAT_ENERGY_GAIN	The quantity of energy gained by an erbast when eating vegetob of a cell
FIGHT_ENERGY_LOSS_E	The quantity of energy loss by an erbast when fighting and surviving a carviz
SPAWN_MAX_E	The maximum number of erbast that can spawn in a Herd
PERC_STARVING_E	The percentage of the self energy at which a erbast start “starving” and needs absolutely food
PERC_STARVING_H	The percentage of the average energy between the members of the group at which a Herd start “starving” and search absolutely for food
MAX_BIRTH_E	The maximum number of offspring that a erbast can generate at the end of his life
MAX_DENSITY	The maximum density a vegetob can spawn in a cell with
GROWTH_RATE	The increment of the vegetob density each day
VEGETOB_EATEN	The decrement of the vegetob density when eaten by an erbast, depending also by the value of energy gained by the erbast.